

**SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE**  
**SUBJECT : CYBER & INFORMATION SECURITY**

**AIM : 1.A:- Design and implement algorithms to encrypt and decrypt messages using classical**

**substitution techniques**

**1. B:- Design and implement algorithms to encrypt and decrypt messages using transposition techniques**

**CODE:**

**1A: CIPHER SUBSTITUTION TECHNIQUE(CLI EDITION)**

```
def caesar_encrypt(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            start = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - start + shift) % 26 + start)
        else:
            result += char
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)

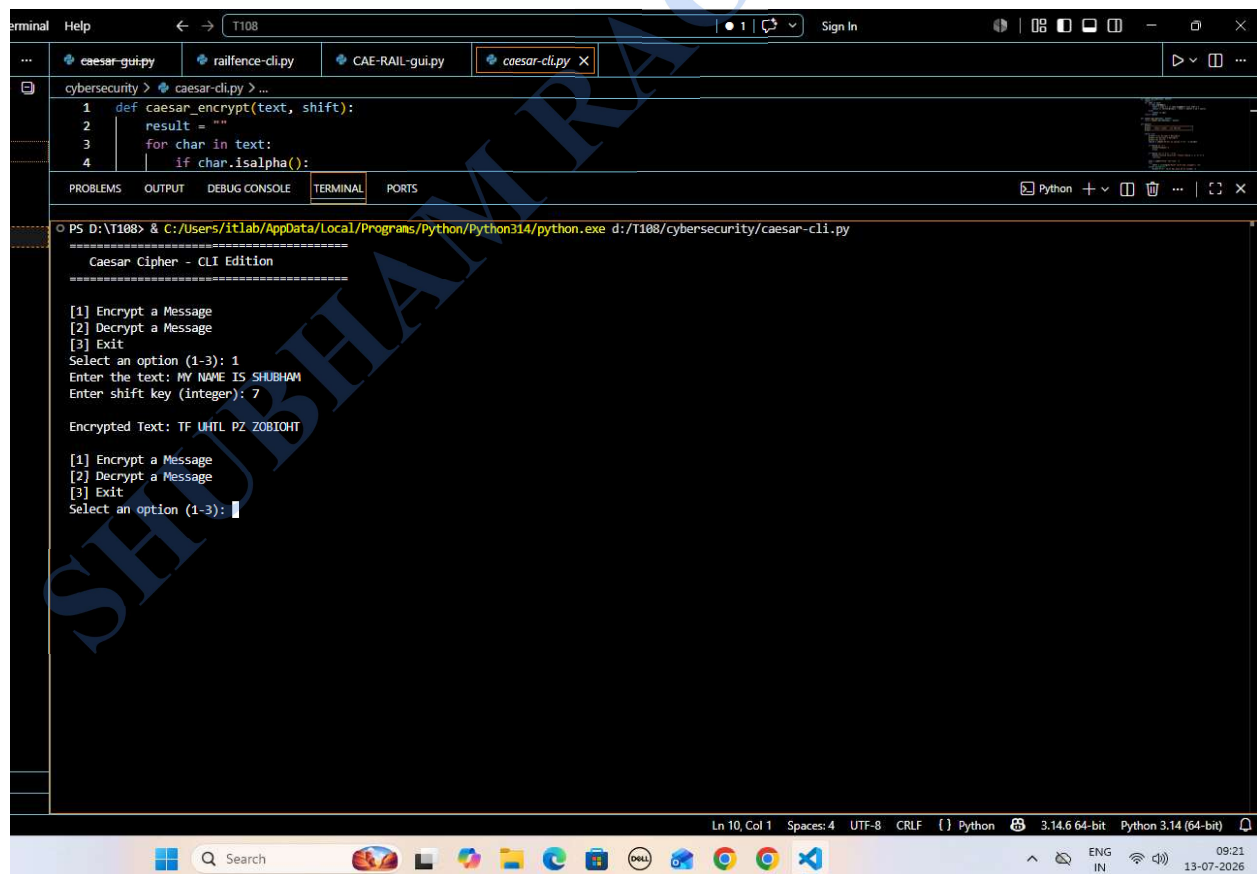
def main():
    print("=====")
    print("    Caesar Cipher - CLI Edition    ")
    print("=====")

    while True:
        print("\n[1] Encrypt a Message")
        print("[2] Decrypt a Message")
        print("[3] Exit")
        choice = input("Select an option (1-3): ").strip()

        if choice == '3':
            print("Goodbye!")
            break

        if choice not in ['1', '2']:
            print("Invalid selection. Please choose 1, 2, or 3.")
```

## OUTPUT



**SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE**  
**SUBJECT : CYBER & INFORMATION SECURITY**

**1B: RAIL FENCE TRANSPOSITION CLI EDITION**

```
def rail_fence_encrypt(text, rails):
    if rails <= 1 or not text:
        return text
    fence = [['\n' for _ in range(len(text))] for _ in range(rails)]
    rail = 0
    direction_down = False
    for i in range(len(text)):
        if (rail == 0) or (rail == rails - 1):
            direction_down = not direction_down
        fence[rail][i] = text[i]
        rail += 1 if direction_down else -1
    return "".join([fence[i][j] for i in range(rails) for j in
range(len(text)) if fence[i][j] != '\n'])

def rail_fence_decrypt(cipher, rails):
    if rails <= 1 or not cipher:
        return cipher
    fence = [['\n' for _ in range(len(cipher))] for _ in range(rails)]
    rail = 0
    direction_down = None
    for i in range(len(cipher)):
        if rail == 0: direction_down = True
        if rail == rails - 1: direction_down = False
        fence[rail][i] = '*'
        rail += 1 if direction_down else -1
    index = 0
    for i in range(rails):
        for j in range(len(cipher)):
            if (fence[i][j] == '*') and (index < len(cipher)):
                fence[i][j] = cipher[index]
                index += 1
    result, rail = [], 0
    for i in range(len(cipher)):
        if rail == 0: direction_down = True
        if rail == rails - 1: direction_down = False
        if fence[rail][i] != '\n':
            result.append(fence[rail][i])
```

**SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE**  
**SUBJECT : CYBER & INFORMATION SECURITY**

```
        rail += 1 if direction_down else -1
    return "".join(result)

def main():
    print("=====")
    print("    Rail Fence Cipher(Transposition) - CLI Edition    ")
    print("=====")

    while True:
        print("\n[1] Encrypt a Message")
        print("[2] Decrypt a Message")
        print("[3] Exit")
        choice = input("Select an option (1-3): ").strip()

        if choice == '3':
            print("Goodbye!")
            break

        if choice not in ['1', '2']:
            print("Invalid selection. Please choose 1, 2, or 3.")
            continue

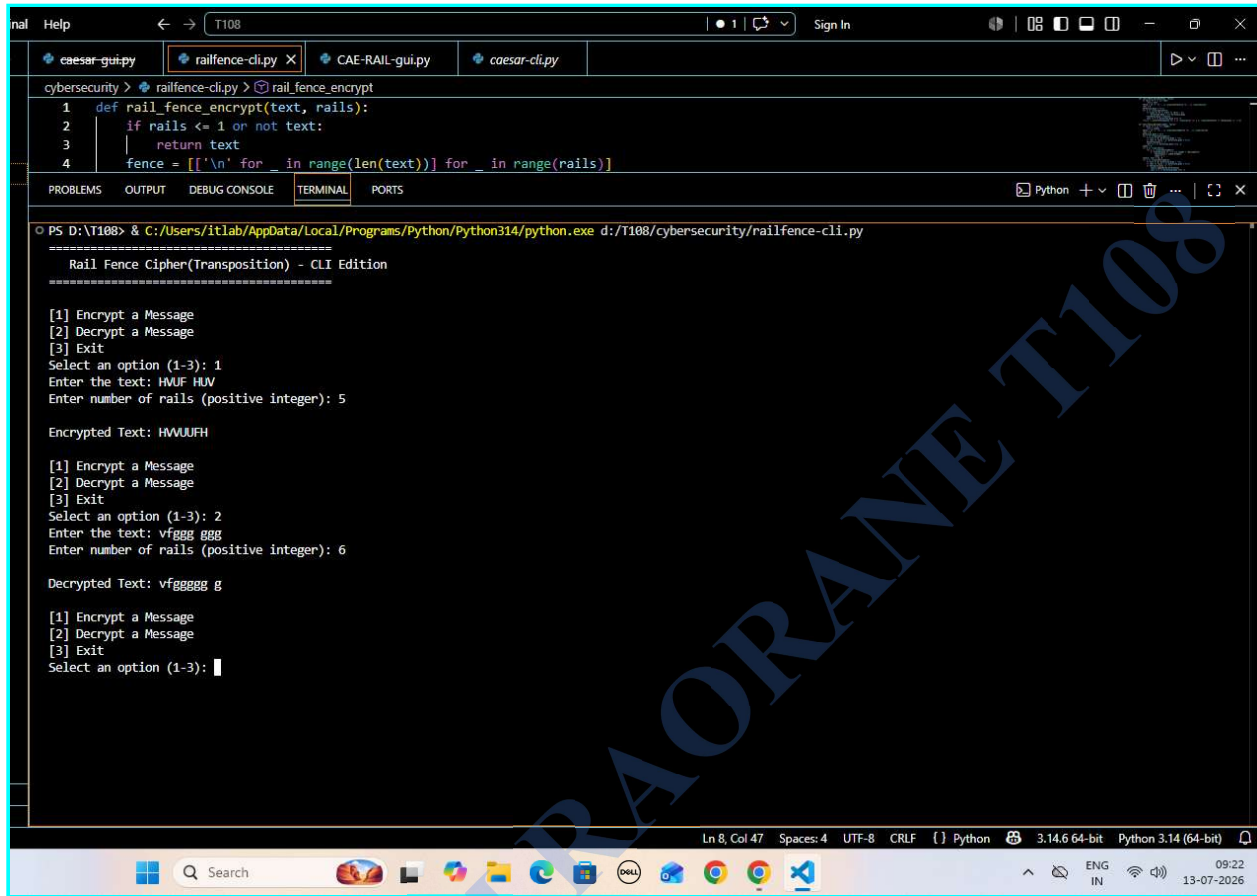
        text = input("Enter the text: ")
        try:
            rails = int(input("Enter number of rails (positive integer): "))
            if rails < 1: raise ValueError
        except ValueError:
            print("Error: Rails must be a positive integer.")
            continue

        if choice == '1':
            print(f"\nEncrypted Text: {rail_fence_encrypt(text, rails)}")
        else:
            print(f"\nDecrypted Text: {rail_fence_decrypt(text, rails)}")

if __name__ == "__main__":
    main()
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE  
SUBJECT : CYBER & INFORMATION SECURITY

OUTPUT



The screenshot shows a VS Code editor with a file named `rail_fence_encrypt` open. The code defines a function `def rail_fence_encrypt(text, rails):` that implements a transposition cipher. Below the code editor, the 'TERMINAL' tab is active, showing the output of running the script. The output displays a menu with options to encrypt, decrypt, or exit. It shows the encryption of the text 'HMUF HMV' with 5 rails, resulting in 'HWUUFH', and the decryption of 'vfeggg g' with 6 rails, resulting in 'vfeggg g'.

```
cybersecurity > rail_fence_encrypt
1 def rail_fence_encrypt(text, rails):
2     if rails <= 1 or not text:
3         return text
4     fence = [['\n' for _ in range(len(text))] for _ in range(rails)]

PS D:\T108> C:\Users\itlab\AppData\Local\Programs\Python\Python314\python.exe d:/T108/cybersecurity/railfence-cli.py

Rail Fence Cipher(Transposition) - CLI Edition
=====

[1] Encrypt a Message
[2] Decrypt a Message
[3] Exit
Select an option (1-3): 1
Enter the text: HMUF HMV
Enter number of rails (positive integer): 5

Encrypted Text: HWUUFH

[1] Encrypt a Message
[2] Decrypt a Message
[3] Exit
Select an option (1-3): 2
Enter the text: vfeggg g
Enter number of rails (positive integer): 6

Decrypted Text: vfeggg g

[1] Encrypt a Message
[2] Decrypt a Message
[3] Exit
Select an option (1-3):
```

GUI CODE:



The screenshot shows a code editor with the following Python code for a GUI application:

```
import tkinter as tk
from tkinter import messagebox
from tkinter import ttk

def caesar_encrypt(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            start = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - start + shift) % 26 + start)
        else:
            result += char
    return result

def caesar_decrypt(text, shift):
```

**SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE**  
**SUBJECT : CYBER & INFORMATION SECURITY**

```
return caesar_encrypt(text, -shift)

def rail_fence_encrypt(text, rails):
    if rails <= 1 or not text: return text
    fence = [['\n' for _ in range(len(text))] for _ in range(rails)]
    rail, direction_down = 0, False
    for i in range(len(text)):
        if rail == 0 or rail == rails - 1: direction_down = not
direction_down
        fence[rail][i] = text[i]
        rail += 1 if direction_down else -1
    return "".join([fence[i][j] for i in range(rails) for j in
range(len(text)) if fence[i][j] != '\n'])

def rail_fence_decrypt(cipher, rails):
    if rails <= 1 or not cipher: return cipher
    fence = [['\n' for _ in range(len(cipher))] for _ in range(rails)]
    rail, direction_down = 0, None
    for i in range(len(cipher)):
        if rail == 0: direction_down = True
        if rail == rails - 1: direction_down = False
        fence[rail][i] = '*'
        rail += 1 if direction_down else -1
    index = 0
    for i in range(rails):
        for j in range(len(cipher)):
            if fence[i][j] == '*' and index < len(cipher):
                fence[i][j] = cipher[index]
                index += 1
    result, rail = [], 0
    for i in range(len(cipher)):
        if rail == 0: direction_down = True
        if rail == rails - 1: direction_down = False
        if fence[rail][i] != '\n':
            result.append(fence[rail][i])
            rail += 1 if direction_down else -1
    return "".join(result)

def launch_app():
```

**SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE**  
**SUBJECT : CYBER & INFORMATION SECURITY**

```
def on_cipher_change(*args):
    if "Caesar" in cipher_var.get():
        key_label_var.set("Shift Key:")
    else:
        key_label_var.set("Number of Rails:")
    process_text()

def process_text(*args):
    text = text_input.get("1.0", tk.END).strip('\n')
    selection = cipher_var.get()
    mode = mode_var.get()

    try:
        key_val = key_var.get()
        key = int(key_val) if key_val else (3 if "Caesar" in selection
else 1)

        if "Transposition" in selection and key < 1: raise ValueError
    except ValueError:
        text_output.config(state=tk.NORMAL)
        text_output.delete("1.0", tk.END)
        text_output.insert(tk.END, "[Error: Valid integer required]")
        text_output.config(state=tk.DISABLED)
        return

    if "Caesar" in selection:
        output = caesar_encrypt(text, key) if mode == "Encrypt" else
caesar_decrypt(text, key)
    else:
        output = rail_fence_encrypt(text, key) if mode == "Encrypt"
else rail_fence_decrypt(text, key)

    text_output.config(state=tk.NORMAL)
    text_output.delete("1.0", tk.END)
    text_output.insert(tk.END, output)
    text_output.config(state=tk.DISABLED)

def copy_to_clipboard():
    root.clipboard_clear()
    root.clipboard_append(text_output.get("1.0", tk.END).strip())
    messagebox.showinfo("Success", "Result copied to clipboard!")
```

**SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE**  
**SUBJECT : CYBER & INFORMATION SECURITY**

```
root = tk.Tk()
root.title("Cryptography Studio")
root.geometry("750x780")

BG_APP = "#F3F4F6"
BG_CARD = "#FFFFFF"
ACCENT = "#4F46E5"
ACCENT_HOVER = "#4338CA"
TEXT_MAIN = "#111827"
TEXT_SUB = "#6B7280"
BORDER = "#D1D5DB"

root.configure(bg=BG_APP)

FONT_H1 = ("Segoe UI", 22, "bold")
FONT_LBL = ("Segoe UI", 10, "bold")
FONT_INPUT = ("Consolas", 12)

style = ttk.Style()
style.theme_use('clam')
style.configure("TCombobox", padding=6, font=("Segoe UI", 10))
style.configure("TEntry", padding=6, font=("Segoe UI", 10))

style.configure("Modern.TButton", font=("Segoe UI", 10, "bold"),
background=ACCENT, foreground="white", borderwidth=0, padding=8)
style.map("Modern.TButton", background=[("active", ACCENT_HOVER)])

header = tk.Frame(root, bg=ACCENT, height=90)
header.pack(fill="x")
header.pack_propagate(False)
tk.Label(header, text="Cryptography Studio", font=FONT_H1, bg=ACCENT,
fg="white").pack(side="left", padx=40, pady=25)

card = tk.Frame(root, bg=BG_CARD, highlightbackground=BORDER,
highlightthickness=1)
card.pack(fill="both", expand=True, padx=40, pady=30)

ctrl_frame = tk.Frame(card, bg=BG_CARD)
ctrl_frame.pack(fill="x", padx=30, pady=(30, 20))
```

**SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE**  
**SUBJECT : CYBER & INFORMATION SECURITY**

```
ctrl_frame.columnconfigure(1, weight=1)

tk.Label(ctrl_frame, text="TECHNIQUE", font=FONT_LBL, bg=BG_CARD,
fg=TEXT_SUB).grid(row=0, column=0, sticky="w", pady=8, padx=(0, 20))
cipher_var = tk.StringVar(value="Caesar Cipher (Substitution)")
cipher_menu = ttk.Combobox(ctrl_frame, textvariable=cipher_var,
values=["Caesar Cipher (Substitution)", "Rail Fence Cipher
(Transposition)"], state="readonly", width=35)
cipher_menu.grid(row=0, column=1, sticky="w", pady=8)
cipher_var.trace_add("write", on_cipher_change)

tk.Label(ctrl_frame, text="MODE", font=FONT_LBL, bg=BG_CARD,
fg=TEXT_SUB).grid(row=1, column=0, sticky="w", pady=8, padx=(0, 20))
mode_var = tk.StringVar(value="Encrypt")
mode_menu = ttk.Combobox(ctrl_frame, textvariable=mode_var,
values=["Encrypt", "Decrypt"], state="readonly", width=15)
mode_menu.grid(row=1, column=1, sticky="w", pady=8)
mode_var.trace_add("write", process_text)

key_label_var = tk.StringVar(value="SHIFT KEY")
tk.Label(ctrl_frame, textvariable=key_label_var, font=FONT_LBL,
bg=BG_CARD, fg=TEXT_SUB).grid(row=2, column=0, sticky="w", pady=8,
padx=(0, 20))
key_var = tk.StringVar(value="3")
key_entry = ttk.Entry(ctrl_frame, textvariable=key_var, width=10)
key_entry.grid(row=2, column=1, sticky="w", pady=8)
key_var.trace_add("write", process_text)

tk.Frame(card, bg=BORDER, height=1).pack(fill="x", padx=30, pady=10)

tk.Label(card, text="INPUT TEXT", font=FONT_LBL, bg=BG_CARD,
fg=TEXT_SUB).pack(anchor="w", padx=30, pady=(15, 5))
text_input = tk.Text(card, height=5, font=FONT_INPUT, relief="flat",
bg="#F9FAFB", highlightbackground=BORDER, highlightthickness=1, padx=10,
pady=10)
text_input.pack(fill="x", padx=30)
text_input.bind("<KeyRelease>", process_text)
```

**SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE**  
**SUBJECT : CYBER & INFORMATION SECURITY**

```
tk.Label(card, text="OUTPUT RESULT", font=FONT_LBL, bg=BG_CARD,
fg=TEXT_SUB).pack(anchor="w", padx=30, pady=(20, 5))

text_output = tk.Text(card, height=5, font=FONT_INPUT, relief="flat",
bg="#EEF2FF", fg=ACCENT, highlightbackground=BORDER, highlightthickness=1,
padx=10, pady=10, state=tk.DISABLED)
text_output.pack(fill="x", padx=30)

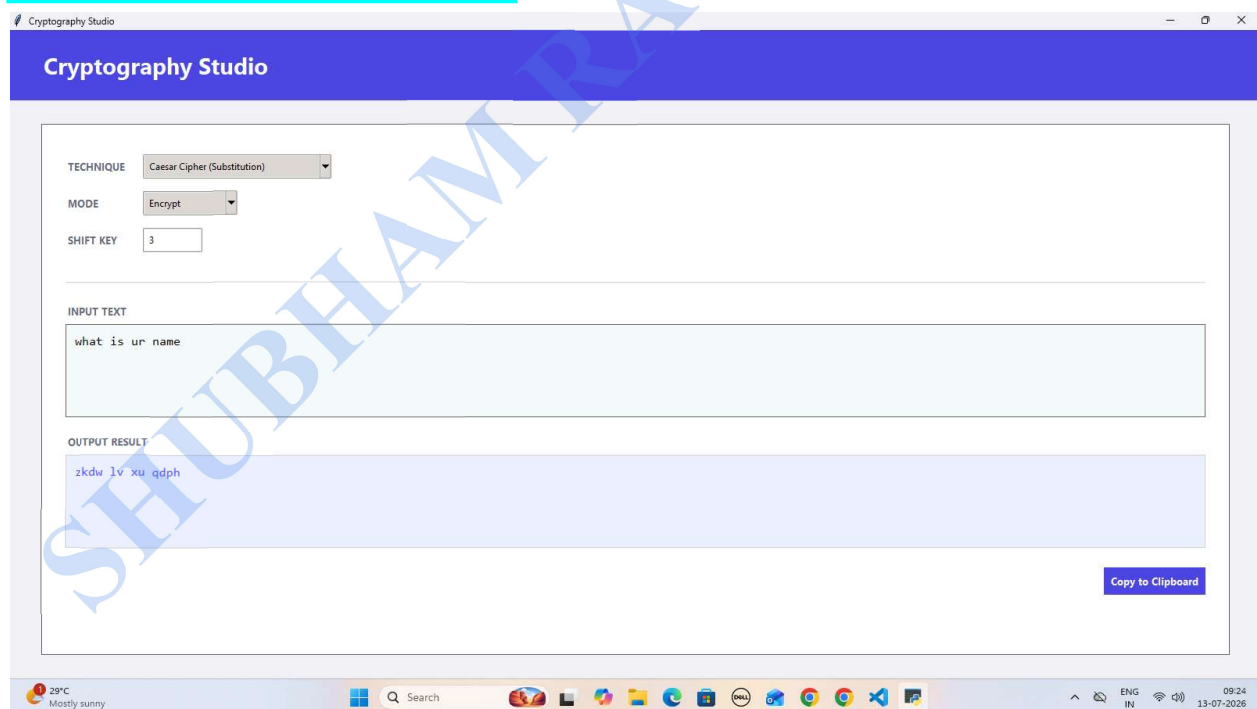
btn_frame = tk.Frame(card, bg=BG_CARD)
btn_frame.pack(fill="x", padx=30, pady=25)
ttk.Button(btn_frame, text="Copy to Clipboard",
style="Modern.TButton", command=copy_to_clipboard).pack(side="right")

process_text()
root.mainloop()

if __name__ == "__main__":
    launch_app()
```

**OUTPUT**

**CAESAR CIPHER SUBSTITUTION**



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE  
SUBJECT : CYBER & INFORMATION SECURITY

RAIL FENCE TRANSPOSITION

Cryptography Studio

TECHNIQUE: Rail Fence Cipher (Transposition)

MODE: Encrypt

Number of Rails: 5

INPUT TEXT: what is ur name |

OUTPUT RESULT: wuh r as etinm a

Copy to Clipboard

29°C Mostly sunny

Q Search

ENG IN 09:25 13-07-2026

CAESAR CIPHER SUBSTITUTION

Cryptography Studio

TECHNIQUE: Caesar Cipher (Substitution)

MODE: Decrypt

SHIFT KEY: 4

INPUT TEXT: what is my name |

OUTPUT RESULT: sdwp eo iu jwia

Copy to Clipboard

29°C Mostly sunny

Q Search

ENG IN 09:31 13-07-2026

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE  
SUBJECT : CYBER & INFORMATION SECURITY

RAIL FENCE TRANSPOSITION

Cryptography Studio

TECHNIQUE: Rail Fence Cipher (Transposition)

MODE: Decrypt

Number of Rails: 9

INPUT TEXT: gfrfgifib gkgmr

OUTPUT RESULT: gifii kmrgggbfgrf

Copy to Clipboard

29°C Mostly sunny

Search

ENG IN 09:32 13-07-2026